

CloudMicroHaskell

Direct-Style Distributed Haskell via Runtime Graph Serialisation

ROBERT KROOK, Chalmers University of Technology and University of Gothenburg, Sweden

LENNART AUGUSTSSON, Unaffiliated, Sweden

Cloud Haskell brings Erlang-style distributed programming to Haskell, but its treatment of mobile code exposes a difficult boundary in the source-level API. Remote processes must be expressed as static closures, messages must satisfy serialisation constraints, and participating nodes are assumed to share the relevant code.

This paper explores a different design point. We present CloudMicroHaskell, a Cloud Haskell-style library built on MicroHaskell, whose runtime represents both code and data as a combinator graph. When a process or message crosses a node boundary, CloudMicroHaskell serialises the reachable graph directly. As a result, remote spawning can be written in direct style: process bodies may capture variables from their surrounding scope, and messages may contain ordinary values, including functions, without programmer-written closure conversion.

We describe the implementation of the CloudMicroHaskell node runtime, including remote spawn, message delivery, monitors, exit propagation, and library implementations of generic servers and supervisors. We evaluate the system with process/message benchmarks, a distributed work-pool benchmark, and a file-synchronisation case study. The results show that runtime graph serialisation makes the Cloud Haskell programming model substantially more direct, while also making the tradeoff explicit: some guarantees enforced by Cloud Haskell's source-level types become dynamic checks, and programmers must be aware of laziness and runtime-owned resources when moving graphs between nodes.

Additional Key Words and Phrases: Partial Evaluation, Combinator Graphs, Haskell, Functional Programming

1 Introduction

Cloud Haskell [4] is a Haskell framework for writing distributed, concurrent programs in the style of Erlang. Erlang's programming model emphasises concurrency, with lightweight processes that share no memory. Instead, inter-process communication is done via message passing. This programming model greatly aids in writing distributed programs that execute on and communicate over a network.

The primary benefit of Cloud Haskell over Erlang is Haskell's strong type system. With it, the API of Cloud Haskell can be described in a very clear and expressive way. It is short and (mostly) elegant. Furthermore, Cloud Haskell extends Erlang's untyped message passing machinery with typed channels, so that communicating processes can exchange messages whose format is known at compile time. Erlang, being

Authors' Contact Information: Robert Krook, krookr@chalmers.se, Chalmers University of Technology and University of Gothenburg, Department of Computer Science and Engineering, Gothenburg, Sweden; Lennart Augustsson, lennart@augustsson.net, Unaffiliated, Gothenburg, Sweden SE-412 96 Gothenburg, Sweden.

dynamically typed, cannot statically enforce a message passing protocol between processes.

These advantages come from making distribution explicit in the API, and that explicitness has practical consequences. The implementation assumes that nodes in the network run a very uniform software environment, and the machinery required to handle closures is necessarily visible to the programmer. We illustrate this with the direct-style program one might like to write

```
1  sendFunc :: Pid -> Int -> ProcessM ()
2  sendFunc p x = send p (\y -> x + y + 1)
```

The above example sends a lambda to a recipient process, which captures a free variable (x). Cloud Haskell uses the type system to enforce that values are serialised before transmission, but note that the type of the function, `Int -> Int`, says nothing about the type of the free variable. It is therefore not possible to provide an ordinary serialiser for functions from the function type alone.

To get around this, a mechanism for `Static` values was devised. Explained briefly, a value is considered static if it is defined at the top-level and refers to only bound variables or other variables defined at the top-level. Furthermore, when a static function is transmitted to a remote node, it must be explicitly closure converted, whereby the environment of the function is populated with its (serialised) free variable. The function is responsible for deserialising its environment itself. To implement the above example, the programmer must write

```
1  sendFunc :: Pid -> Int -> ProcessM ()
2  sendFunc p x = send p clo
3  where
4    -- closures can be serialised
5    clo :: Closure (Int -> Int)
6    clo = MkClosure (static sfun) (encode s)
7
8  sfun :: ByteString -> Int -> Int
9  sfun bs y = let x = decode bs
10             in x + y + 1
```

The previously intuitive and brief program has now been rewritten in a style with a significantly higher cognitive cost. Furthermore, a static value is assumed to exist on both machines (the one running the sending process and the one running the receiving process), and a closure only contains a static reference to it (together with its environment). While it seems like a function is being sent, what is actually being sent is a static reference to it and its environment.

While this is a clever solution to a tricky problem, we note that there are strict requirements placed on participating machines in the network. Namely, that they must run binaries that contain the same static symbols.

In this paper, we ask what Cloud Haskell might look like if the serialisation boundary were moved out of the source-level API and into the runtime representation of the program. We present CloudMicroHaskell, an exploration of the Cloud Haskell programming model using the MicroHaskell compiler [2]. MicroHaskell is a Haskell compiler with a much smaller and more modest runtime system than GHC, the de facto Haskell compiler. MicroHaskell emits a *combinator graph* as its object code — a self-modifying byte code that represents both a program’s data and code. While the cost of this decision is a significant reduction in performance, there are several benefits, which we hope to illustrate with this paper.

Using MicroHaskell changes the tradeoffs in several useful ways, namely

- We use a simple compiler primitive to serialise values of (almost) any kind, including functions. The first variant of `sendFunc` in the introduction is directly runnable in CloudMicroHaskell.
- Our implementation does not require every node in the network to agree beforehand on which static values should exist. Instead, nodes need only agree on a compatible CloudMicroHaskell runtime and the combinator format understood by that runtime.
- CloudMicroHaskell can serialise and transmit a message of (almost) any type, without requiring a serialisable constraint. This reduces the boilerplate required to prepare a data type for transmission over a network.

This does not remove the distribution boundary, but relocates it. The price is that this boundary is no longer enforced by Haskell’s source-level types. Deserialisation is necessarily a dynamic claim that the received graph has the type expected by the receiver. If that claim is wrong, the program may fail at runtime. In that sense, CloudMicroHaskell trades some of Cloud Haskell’s static safety for a much simpler programming model.

2 Programming in CloudMicroHaskell

In this section we present CloudMicroHaskell. At the surface, CloudMicroHaskell looks much like Cloud Haskell. Programs create processes, identify remote nodes, and communicate by message passing. This is deliberate. The point of CloudMicroHaskell is not to propose a new process calculus, but to explore what happens when the serialisation boundary in Cloud Haskell is moved out of the source-level API and into the MicroHaskell runtime.

The most important differences therefore appear in the two operations that move computation and data between nodes, `spawn` and `send`. In Cloud Haskell, these operations expose distribution through `Closure` and `Serializable` constraints. In Cloud-MicroHaskell, these operations accept ordinary process computations and ordinary values, relying instead on MicroHaskell’s graph serialiser (the serialiser and deserialiser are described in section 3).

We therefore focus on these two operations in the main text. The surrounding API follows the familiar Cloud Haskell and Erlang shape¹.

The core difference is visible in the types

```
1  spawn :: NodeId -> ProcessM () -> ProcessM Pid
2  send  :: Identifiable process => process -> a -> ProcessM ()
```

In Cloud Haskell, remote spawning requires a `Closure` and message passing requires a `Serializable` constraint. In CloudMicroHaskell, the programmer supplies the computation or value directly. If the target process or message crosses a node boundary, the runtime will serialise the reachable MicroHaskell graph, capturing both the code and environment. This makes direct-style examples possible to execute, but it also means that some errors that Cloud Haskell eliminates via the type checker become runtime failures in CloudMicroHaskell.

The serialisation primitive is intentionally very broad, but not universal. Some things are owned by a particular runtime instance, such as `MVars` and open file handles. These cannot be serialised. The exceptions are the standard handles `stdin`, `stdout`, `stderr`, and the *null pointer*.

The example below illustrates how `spawn` is used to spawn a concurrent process that captures free variables from the outer process

```
1  data PingPong = Ping | Pong
2
3  f :: ProcessM ()
4  f = do
5    s <- self
6    n <- node
7    n' <- (head . filter (/= n)) <$> nodes
8
9    child <- spawn n' $ do
10      Ping <- expect
11      liftIO $ putStrLn "child received ping!"
12      send s Pong
13
14    send child Ping
15    Pong <- expect
16    liftIO $ putStrLn "parent received pong!"
```

The parent process first fetches its own process identifier by calling `self`. It then spawns a child process on a remote node. The child refers to `s`, which is bound in the parent. When `spawn` crosses the node boundary, the child computation is serialised together with this reachable environment. Additionally, we can send the `PingPong` data type without saying how to serialise it.

¹The full API is described in appendix A

To make it even clearer that code is serialised and sent to a remote node, we show yet another example

```

1  f :: ProcessM ()
2  f = do
3    s <- self
4    n <- node
5    n' <- (head . filter (/= n)) <$> nodes
6
7    let interesting x = x `mod` 7 == 0 && x `mod` 11 == 0
8
9    _ <- spawn n' $ do
10      let r = length (filter interesting [1..1000000])
11      r `seq` send s r
12
13    r <- expect :: ProcessM Int
14    liftIO $ print r

```

We define a local function `interesting :: Int -> Bool`, and reference it from the spawned process. `interesting` will be included in the serialised graph that is sent to the remote node, and reconstructed on that node's heap. The call to `seq` is intentional. Since `CloudMicroHaskell` serialises graphs, not values produced by a `Binary` encoder, sending `r` without first evaluating it could send a thunk back to the original node. The result would still be correct, but the work would happen in the wrong place.

It is not just `spawn` that can be used to send functions to remote processes. `send` uses the same underlying serialiser, and will happily send functions. Consider the example below

```

1  data Apply = Apply (Int -> Int) Int Pid
2
3  worker :: ProcessM ()
4  worker = do
5    Apply f x replyTo <- expect
6    let y = f x
7    y `seq` send replyTo y
8
9  example :: ProcessM Int
10 example = do
11   me <- self
12   n <- node
13   n' <- (head . filter (/= n)) <$> nodes
14
15   workerPid <- spawn n' worker
16
17   let offset = 1

```

```

18   send workerPid (Apply (\x -> x + offset) 41 me)
19
20   expect

```

The message sent to `workerPid` contains a function, and that function captures `offset`. In Cloud Haskell, this is exactly the case that requires static closures and explicit environment serialisation. In CloudMicroHaskell, the function is just part of the message graph. The receiver expects an `Apply` message, extracts the function, and applies it.

The rest of the library follows the familiar process-programming structure. Nodes can be connected and queried; processes have identifiers and mailboxes; selective receive inspects messages by runtime type; monitors and exits make process failure observable; and a node-local registry lets long-running processes be found by name. These operations are not the novelty of CloudMicroHaskell, but they are necessary to make the core `spawn` and `send` design usable in larger programs. Their full API is listed in appendix A.

2.1 Process Behaviours as Libraries

The examples above use `spawn` and `send` directly. To test whether this style scales beyond our toy examples, we implement two familiar Erlang/OTP behaviours as ordinary CloudMicroHaskell libraries, generic servers and supervisors. These behaviours are not built into the runtime, but are implemented using the same process creation, message passing, monitors, and exits available to the programmer.

A generic server captures a common pattern whereby a process responds to requests and then enters a state where it is ready to respond to a subsequent request. Furthermore, the process may maintain a state which may be modified by a request, and the modified state must be used in subsequent requests. The application-specific part is described by a record of callbacks

```

1  data ServerSpec st callReq castReq reply = ServerSpec
2  { setup      :: ProcessM st
3  , handleCall :: st -> callReq -> ProcessM (st, reply)
4  , handleCast :: st -> castReq -> ProcessM st
5  , tearDown   :: st -> ProcessM ()
6  }
7
8  startServer :: NodeId -> ServerSpec st callReq castReq reply
9              -> ProcessM (Server callReq castReq reply)
10 -- run the server in the *current* process
11 runServer :: ServerSpec st callReq castReq reply -> ProcessM ()
12
13 call :: Server callReq castReq reply -> callReq -> ProcessM reply
14 cast :: Server callReq castReq reply -> castReq -> ProcessM ()

```

`startServer` can start the server on a remote node, even though the server specification contains ordinary Haskell functions. Internally, a server loop is spawned together with the supplied callback. The programmer only implements the state transition logic, and the library handles the receive loop, reply routing, and state threading.

For example, a counter server can be described by giving its initial state and two handlers

```

1  data CallRequest = Get
2  data CastRequest = Inc | Reset
3
4  counterSpec :: ServerSpec Int CallRequest CastRequest Int
5  counterSpec = ServerSpec
6    { setup      = return 0
7    , handleCall = \st Get -> return (st, st)
8    , handleCast = \st req ->
9      case req of
10         Inc  -> return (st + 1)
11         Reset -> return 0
12    , tearDown  = \_ -> return ()
13    }

```

A client can then start the server remotely and interact with it through a typed handle:

```

1  do counter <- startServer n counterSpec
2     cast counter Inc
3     cast counter Inc
4     i <- call counter Get
5     liftIO $ print i

```

Supervisors capture a second common pattern, where a supervisor process keeps a set of child processes alive according to a restart policy. Again, the implementation is library code. A supervisor starts its children using `spawn`, monitors them, and reacts to `ProcessDied` messages by consulting its policy.

```

1  data ChildSpec = ChildSpec
2    { childName  :: String
3    , childStart  :: ProcessM ()
4    , childNode   :: Maybe NodeId
5    , childRestart :: Restart
6    }
7
8  data Restart = Permanent | Transient | Temporary
9  data Strategy = OneForOne | OneForAll
10

```

```

11 data SupervisorSpec = SupervisorSpec
12   { strategy :: Strategy
13   , intensity :: Int
14   , period    :: Int
15   , children  :: [ChildSpec]
16   }
17
18 supervise :: SupervisorSpec -> ProcessM Pid
19 getChild  :: Pid -> String -> ProcessM (Maybe Pid)

```

The field `childStart` is an ordinary `ProcessM ()`. It may be a generic server, another supervisor, or an application-specific process. If `childNode` names a remote node, the child is spawned there. If the child dies and the restart policy says it should be restarted, the supervisor simply spawns the same computation again.

For instance, a supervised remote counter server is just a child whose start action runs the generic server

```

1 do let child = ChildSpec
2     { childName  = "counter"
3     , childStart = runServer counterSpec
4     , childNode  = Just n
5     , childRestart = Permanent
6     }
7
8 sup <- supervise SupervisorSpec
9   { strategy = OneForOne
10   , intensity = 3
11   , period    = 5000
12   , children  = [child]
13   }
14
15 Just pid <- getChild sup "counter"
16 let counter = serverFromPid pid
17 cast counter Inc

```

This is the role of the Erlang-style parts of `CloudMicroHaskell` in this paper. They are evidence that the direct `spawn` and `send` interface is expressive enough to build structured distributed programs, while keeping the central tradeoff visible: process behaviour is easy to move because it is represented as a runtime graph, and the safety of that movement is correspondingly checked at runtime.

3 Implementation

3.1 MicroHs & Graph Reduction

The Haskell implementation used in this paper is MicroHaskell [2] (MicroHs for short). What matters for our purposes is not MicroHs itself, but the runtime support it provides: the serialisation primitives that make the framework work. The framework is therefore not tied to MicroHs, and could be used unchanged with another Haskell compiler whose runtime provides the same primitives.

Serialisation & Deserialisation. MicroHs uses graph reduction with combinators [13]. This means that a running program is represented by a graph in memory that is continually rewritten. The leaf nodes in the graph are the combinators and the interior nodes are application nodes. There is sharing (which is necessary for lazy evaluation) and cycles (because of recursion). To serialise an arbitrary value, you simply have to traverse the sub-graph reachable from the starting node, *i.e.* the value to serialise. With a little care it is not difficult to use a two-pass algorithm that preserves sharing and cycles in the graph. MicroHs uses a simple textual format that is portable between different platforms. Deserialisation is also mostly straightforward.

The API of these primitives is

```
1  serialize  :: a -> IO ByteString
2  deserialize :: ByteString -> IO a
```

Serialisation will throw an exception if the value cannot be serialised, *e.g.*, if it contains a C pointer or an `MVar`.

The deserialisation type signature is, of course, a lie. A `ByteString` cannot be deserialised as any type one chooses; it represents one specific type. Trying to use it in any other way will result in a runtime error.

These are the low-level primitives. On top of this you can build something more type-safe, *e.g.*, the `ByteString` can be prefixed by some type descriptor (like a hash of the types involved) to make sure the serialiser and deserialiser agree. So a higher-level API would be

```
1  serializeHL :: TypeDescribable a => a -> IO ByteString
2  deserializeHL :: TypeDescribable a => ByteString -> IO (Maybe a)
```

Note that while the type of the message has a constraint, inside the serialised value all kinds of unconstrained types can be used. All we need is that the sender and receiver agree on how to make the type descriptor.

For messages sent over the Internet, using some encryption, *e.g.*, SSL [8], would be advisable.

Other implementations. This kind of serialisation is not easy for an implementation that compiles to machine code (like GHC), since you do not want to send binary code. But it is possible if the implementation uses some kind of bytecode and a JIT compiler. In this case the bytecode will be part of the serialised data and can be JIT-compiled

again by the receiver. This is how the Mu implementation at Standard Chartered worked [1].

3.2 Node Runtime Architecture

A CloudMicroHaskell program always runs a *node*. The node is invoked from main with a `NodeConfig`, a structure holding the node's identity (an IP address and port number). The Haskell main thread creates an initial *node state*, spawns a *TCP listener* thread that awaits incoming connection requests, and then enters the main node loop. After this initial setup, the node continuously awaits commands from either a remote peer over the network, or from a process running on the same machine.

This gives the runtime a simple shape, where processes and peer-reader threads do not manipulate the distributed runtime directly. Instead, they communicate with the node loop by submitting commands. The rest of this subsection explains that boundary: which requests can arrive at a node, where they come from, and how the node turns them into local runtime effects.

We first conclude that the node clearly needs to be able to process commands. Examples of commands could be *install this monitor*, *send this message*, *connect to this remote node*, to name just a few. Some of the commands the node needs to process come from processes that are running on the node in question, whereas some commands will be sent from remote nodes over the network. We model the commands without much flair

```

1  -- | A command for the node to handle
2  data NodeCommand
3    = Local Command      -- ^ A command from a process on this node
4    | NonLocal PeerCommand -- ^ A command sent from another node
5
6  -- | Commands that a local process sends to the node
7  data Command where
8    Connect :: SockAddr -> Maybe (MVar ()) -> Command
9    Deliver :: Pid -> Mail -> Command
10   -- more
11
12  -- | A command that a remote node has asked the node to execute
13  data PeerCommand
14    = PeerConnect SockAddr Socket [SockAddr]
15    | PeerDeliver Pid Mail
16   -- more

```

The node processes commands as they arrive in an `cmdMVar :: MVar [NodeCommand]` maintained in its node state. We choose to represent the queue of commands as a list for simplicity, but observe that another data structure would be more efficient. Furthermore, different kinds of commands could be given different priorities, but in our implementation they all have equal priority.

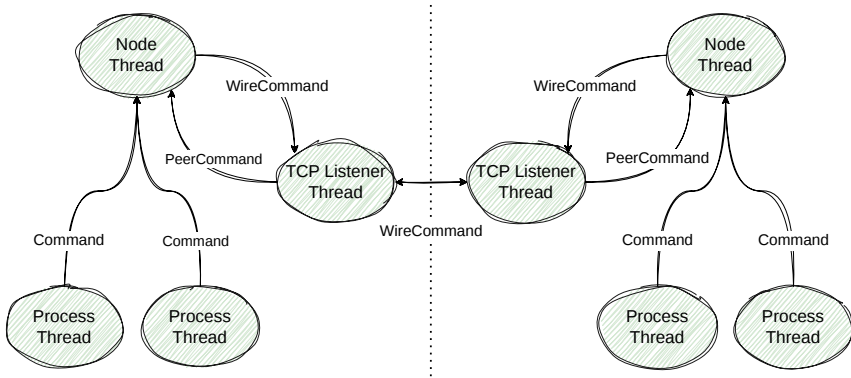


Fig. 1. The figure illustrates how the different types of commands flow through the node application. Every node in this diagram is a Haskell green thread, and the dotted line represents network communication. Process threads will send the node ordinary `Command` values, whereas the node thread needs to convert such a command to an appropriate `WireCommand` before the TCP listener thread can send it to a remote node. A TCP listener converts a received `WireCommand` to a `PeerCommand` before sending it to the node thread.

A local process will construct a `Local Command` and write it to the `MVar`, whereas a remote command will be received by a green thread monitoring a socket, as a `NonLocal PeerCommand`. While a command is travelling over the network, it is represented as yet another type of command, `WireCommand`.

```

1 data WireCommand
2   = WireConnect SockAddr [SockAddr]
3   | WireDeliver Pid Int Int
4   -- more
  
```

During transit, a command will have been turned primarily into a buffer of bytes. In the example constructors above, the `WireDeliver` variant carries a process identifier, and two sizes, denoting the length of the two following data packets (a serialised `TypeRep`, and the `Mail` itself). The TCP listener will convert a `WireCommand` to a `PeerCommand` by e.g. reading potential buffers and reconstructing values that have been serialised.

The node processes commands in the order in which they arrive. A figure that shows how the different types of commands flow through the node application is shown in fig. 1. The command interface is what lets the node remain the owner of the runtime state. Mailboxes, monitors, connections, pending receives, and the name registry are not exposed to processes directly, but are rather updated only by the node loop while handling commands. Instead of showing the entire node state definition at once, we will introduce the relevant pieces as they become necessary.

3.3 The ProcessM Monad

The `ProcessM` monad is a central part of `CloudMicroHaskell`, as it models a *process*. Conceptually, a process is an independent entity that performs computations regardless of what other processes do, and communicates with other processes only via message passing. It is modelled as a function that takes the node state and its own pid, and then produces an `IO` action.

```
1 data ProcessM a = ProcessM { unProcessM :: NodeState -> Pid -> IO a }
```

The process needs to know about the node state as the node state supplies, most notably, the `MVar` through which commands are sent. A common pattern in `ProcessM` primitives that needs to wait for a result from the node is that they allocate a fresh `MVar`, send its command to the node (which includes the `MVar`), and then block until the result is there. An example is shown below

```
1  -- | Register a Pid under a name
2  register :: Pid -> String -> ProcessM ()
3  register pid name = ProcessM $ \ns _ -> do
4      -- create a fresh MVar to block until the node has registered the name
5      mv <- newEmptyMVar
6
7      -- send the command to the node
8      modifyMVar_ (cmdMVar ns) $ \cmds -> return (cmds ++ [Local (Register pid name mv)])
9
10     -- wait for the node to write to the MVar, indicating that it is done
11     takeMVar mv
```

Additionally, functions that return a result fetched from the node state must force it. Consider the function `node` that returns the node a process is currently executing on.

```
1 node :: ProcessM NodeId
2 node = ProcessM $ \ns _ ->
3     let nd = NodeId $ nodeAddr ns
4     in rnf nd `seq` return nd
```

If we did not force the evaluation of the `NodeId`, there is a high risk that the result from `node` would be an unevaluated thunk. We illustrate the problem with this with the code snippet below, which shows a process that spawns another process that references the fetched `NodeId`.

```
1 do n <- node
2   ns <- filter ((/=) n) <$> nodes
3   mapM_ (remoteNode -> spawn remoteNode (spawn n (liftIO $ putStrLn "hi"))) ns
```

The above program fetches the current process's node, and then spawns a process on every other remote node. That process in turn spawns a process on the original node again, by referencing the free variable `n`. If `n` was an unevaluated thunk that

references the node state, the outer call to `spawn` would not succeed, as the serialiser would follow the graph into the node state and try to serialise its inner `MVars`. By ensuring the result is forced, we know that the node state is no longer referenced by `n` and serialising proceeds as expected.

This is also where the conceptual process and the runtime bookkeeping part ways. A `ProcessM` computation is given a `Pid`, but the resources associated with that `Pid` are owned by the node. The process has a mailbox, may have pending receives, may be registered under a name, and may be monitored by other processes. However, none of these are fields of the process itself. They are entries in the node state, and the process can affect them only by sending commands to the node.

This distinction is important for distribution. When a process is spawned remotely, we do not serialise its mailbox, monitors, or other local runtime resources. We serialise only the computation to run. The receiving node allocates a fresh `Pid`, creates the corresponding local runtime entries, and then runs the computation with its own node state.

3.4 Remote Spawn

The central contribution of CloudMicroHaskell is that `spawn` can take an ordinary `ProcessM` computation and run it on another node. We now describe how a call to `spawn` moves through the node runtime, crosses the network, and results in a fresh green thread running the supplied computation on a remote machine. The example program we begin with is shown below

```

1  do n <- node
2      anotherNode <- (head . filter ((/=) n)) <$> nodes
3
4      let x = 7
5
6      _ <- spawn anotherNode $ do
7          liftIO $ putStrLn $ show x
8          let x = 10
9          liftIO $ putStrLn $ show x
10
11     return ()

```

We note that the spawned process references the outer `x`, a free variable.

The implementation of `spawn` will, briefly explained, send this request as a command to the node. As `spawn` returns the `Pid` of the spawned process, it blocks until the remote node has created the process and acknowledged its `Pid`. A fresh `MVar` is allocated where the node will write the `Pid` once it receives it together with a spawn confirmation from the remote node. Additionally, a unique token is generated and paired with the spawn request, so that an acknowledgement of another spawn request is not confused with the current request. The result `MVar` is associated with the unique token in a mutable map reachable from the node state.

```

1 spawn :: NodeId -> ProcessM () -> ProcessM Pid
2 spawn (NodeId targetAddr) action = ProcessM $ \ns _myPid -> do
3   -- generate token and create MVar, to record in the node state
4   reqId   <- newUniqueToken
5   replyVar <- newEmptyMVar
6   modifyIORef' (spawnRequests ns) (Map.insert reqId replyVar)
7
8   -- send the spawn command to the node loop
9   modifyMVar_ (cmdMVar ns) $ \cmds ->
10    return (cmds ++ [Local (SpawnOn targetAddr reqId (unProcessM action))])
11
12   -- block here until the node loop publishes the process identifier of the spawned process
13   pid <- takeMVar replyVar
14   modifyIORef' (spawnRequests ns) (Map.delete reqId)
15
16   return pid

```

When the node processes the `SpawnOn` command, it checks whether the node the process should be spawned on is the current node or a remote one. We omit the handling of a local spawn for now, and focus on the remote spawn instead. When the remote node spawns the requested process, the behaviour of the loop is the same as when doing the local spawn, so the reader can hopefully infer how it is implemented.

```

1 -- this function pattern matches on many commands, but we just show SpawnOn here
2 handleLocalCommand :: NodeState -> Command -> IO ()
3 handleLocalCommand ns (SpawnOn targetAddr reqId f) = do
4   if targetAddr == nodeAddr ns
5     then do pid <- startLocalProcess ns f
6              completeSpawnRequest ns reqId pid
7     else do c <- getConn ns targetAddr
8              sendSpawn c (nodeAddr ns) reqId f

```

The node loop fetches the connection object associated with the remote node, an open socket. It then calls the `CloudMicroHaskell` network layer to transmit the `ProcessM` value.

```

1 -- remember that the ProcessM monad wrapped a function of type
2 -- NodeState -> Pid -> IO a
3 sendSpawn :: Socket -> SockAddr -> Int -> (NodeState -> Pid -> IO ()) -> IO ()
4 sendSpawn conn spawnerAddr reqId f = do
5   (buf, len) <- enc f -- serialise the ProcessM function
6
7   -- send first the serialised WireSpawn command, and then the serialised graph
8   sendLine conn (show (WireSpawn spawnerAddr reqId len))
9   sendAll conn buf len

```

```

10
11     free buf

```

Here is where we invoke our magic serialisation function. The serialised graph will reference everything that was reachable from the expression, including the outer free variable. This is now a serialised function that contains both its code and environment.

After serialisation, we get a pointer to a memory buffer holding our ready-to-send bytes. The `WireCommand` that we send to the remote node includes the unique token that we have associated with the spawn request, as well as the length of the following data buffer. The remote node's TCP listener will receive first the `WireCommand`, look at it to figure out how many bytes to read for the subsequent graph, and then decode it when it turns it into a `PeerCommand`. We omit most of the listener code, but show how the received `WireSpawn` message is treated

```

1  handleWireCommand (WireSpawn spawnerAddr reqId payloadLen) = do
2    buf <- mallocBytes payloadLen
3    ok <- recvExact conn buf payloadLen
4
5    if not ok
6      then do
7        -- error handling code, omitted for brevity
8      else do
9        f <- dec buf payloadLen :: IO (NodeState -> Pid -> IO ())
10       free buf
11       modifyMVar_ (cmdMVar ns) $ \cmds ->
12         return (cmds ++ [NonLocal (PeerSpawn spawnerAddr reqId f)])

```

After the `WireSpawn` command has already been parsed, we know that we should read a subsequent buffer of data, the graph. We allocate a buffer of the appropriate length and read exactly that many bytes into it. We then invoke our magic deserialiser, which reconstructs the graph on the heap of the remote CloudMicroHaskell node. Here we annotate the deserialiser with the type of the function that `ProcessM` wraps. There is no verification that the parsed graph represents a value of that specific type, so we must be very careful to properly line up the types of the values we serialise, and the values we deserialise. We then enqueue the `PeerSpawn` command with the node loop.

Now we are almost home. The node loop on the remote node must invoke its `PeerCommand` handler to deal with the command. It starts the local process, and sends the `Pid` together with the unique token back to the original node. We omit the code for `startLocalProcess`, but mention that it uses `forkIO` to spawn a green thread that runs the `ProcessM` function we just deserialised. The process's mailbox and associated metadata are then added to the node state.

```

1  handleNonLocalCommand ns (PeerSpawn spawnerAddr reqId f) = do

```

```

2  pid <- startLocalProcess ns f
3  c    <- getConn ns spawnerAddr
4  sendLine c (show (WireSpawnAck reqId pid))

```

At the wire level the original node receives the `WireSpawnAck`, which the listener promptly converts into a `PeerSpawnAck`. The spawn request is then completed by invoking the `completeSpawnRequest` function that was briefly shown in the code that described how the node handled the initial spawn request. `completeSpawnRequest` publishes the `Pid` into the `MVar` that is associated with the unique token. This unblocks the process that initiated the spawn request, which now has access to the process identifier of the new process. This concludes the handling of `spawn`.

3.5 Message Passing

The underlying mechanism for transmitting messages between communicating processes is the same. As values are, like code, just subgraphs in the combinator graph, the serialiser and deserialiser work here as well. If a message has to traverse the network it will be encoded together with a runtime representation of its type. If a message does not have to traverse the network, and is being sent to a process running on the same node, the message is passed by its pointer into the graph. We capture these two kinds of messages in the following data type

```

1  data Mail where
2    HeapMail  :: a -> Mail
3    EncodedMail :: TypeRep -> Ptr Word8 -> Int -> Mail

```

We use `TypeRep` from the `Typeable` type class to observe the types of values at runtime. Both GHC and MicroHaskell derive `Typeable` instances for every type, so the programmer does not need to implement them.

Every process has a mailbox where its messages go, but the mailbox is owned by the node loop. The code for `send` itself is fairly brief

```

1  send :: Identifiable i => i -> a -> ProcessM ()
2  send targetPid x = ProcessM $ \ns _ -> do
3    targetPid' <- toPid targetPid ns
4    modifyMVar_ (cmdMVar ns) $ \cmds ->
5      return (cmds ++ [Local (Deliver targetPid' (HeapMail x))])

```

As before, it sends a command to the node loop. There are two types that are `Identifiable`, `Pid` and `String`. `toPid` does a lookup in an `IORef` fetched from the node state if the instantiated type is `String`.

The node loop will dispatch different behaviour depending on whether the message has to traverse the network or not.

```

1  handleLocalCommand ns (Deliver targetPid mail) = do
2    let Pid (NodeId targetAddr) _ = targetPid
3    if targetAddr == nodeAddr ns
4      then deliverMailLocal ns targetPid mail

```



```
5   else deliverMailRemote ns targetPid mail
```

We omit the code for `deliverMailLocal`, but say that it is responsible for delivering the message to the mailbox of the receiving process. If the receiving process is already blocked waiting for a message, it will first see whether this message could be delivered, and only if not is the mail inserted in the mailbox.

When delivering a message to a remote process on another node, the message must be encoded and transmitted. We note that we transmit not just the serialised value, but also the serialised `TypeRep`. The receiving node will figure out which mailbox to place it in, after which the send is finished.

Forcing values before sending. Something to keep in mind when sending messages between processes is that messages will be sent as they are, then and there. The serialiser walks the combinator graph, and if a value is still an unevaluated thunk, the unevaluated thunk will be serialised and transmitted. This might be the intended behaviour, but it will most likely be the intent of the programmer that an evaluated value should be sent. The problems with sending unintended unevaluated thunks are twofold; firstly, a seemingly small expression might make a large part of the sender's heap be serialised and sent to the remote node. Secondly, if we are using processes and message passing to achieve speedups by distributing our computation over a network, work might not be performed on those nodes where the programmer expects it to be done. We note that in Cloud Haskell, this concern does not arise. The `Binary` instance required by Cloud Haskell's `send` requires a value to be evaluated as it is serialised.

This is the same discipline familiar to programmers of parallel Haskell. A spark created by `par` only buys parallelism if the sparked expression is itself evaluated by another core. In CloudMicroHaskell, the analogous failure mode is not lost parallelism but misplaced computation, and possibly a much larger message than the programmer intended. The remedy is the same: ensure the message is in normal form before handing it to `send`, using `seq`, bang patterns, or a deep-force such as `deepseq`.

3.6 Failure Detection and Exit Propagation

The fault-tolerance primitives in CloudMicroHaskell are built around process death detection. The runtime does not try to recover a failed process by itself, but instead makes failure observable to other processes. Application-level code, such as supervisors, can be programmed to decide whether a process should restart or not.

One process can observe the death of another by, as described previously, installing a monitor. As the node owns all the state, it maintains a table which maps each watched process to the processes that watch it, and the monitor action each watcher has requested.

```
1   monitors :: IORef (Map.Map Pid [(Pid, MonitorAction)])
```

Unsurprisingly, a monitor is registered by sending a command to the node loop. If the process that should be watched is local, the node records the monitor in its

monitors table. Instead, if the process lives on another node, the request is sent to the remote node as a `WireMonitor` command. The remote node will record the monitor in its own monitors table. The short summary is that a node is responsible for detecting process death and recording which watchers must be notified.

Every `CloudMicroHaskell` process is implemented by a Haskell green thread. When a process is started, the runtime forks the thread with a finaliser. This finaliser runs regardless of whether the process returns normally, calls `terminate`, dies because of an exception, or is killed by another process via `exit`.

```

1  startLocalProcess :: NodeState -> String -> (NodeState -> Pid -> IO ()) -> IO Pid
2  startLocalProcess ns label f = do
3    pid <- allocatePid ns
4    tid <- forkFinally (f ns pid) $ \res -> do
5      let reason = reasonFromResult res
6      cleanupProcess ns pid
7      notifyMonitors ns pid reason
8    modifyIORef' (threadIds ns) (Map.insert pid tid)
9    return pid

```

The finaliser turns the result of the thread into an `ExitReason`. Normal return and `terminate` become `ExitNormal`, whereas termination by `exit` preserves the `ExitReason` supplied by the caller. An uncaught exception becomes `ExitOther`. The runtime will then remove the process-local state such as its mailbox, pending receives, registered name, and thread identifier, before finally notifying the monitors.

Notification is also routed through the node loop. For each registered watcher the node loop checks whether it is a local process or not. A local watcher is notified immediately, whereas a remote watcher is notified by sending a `WireProcessDied` command to the watcher's node.

```

1  applyMonitorAction ns watcherPid watcheePid TrapExit reason =
2    deliverMailLocal ns watcherPid (HeapMail (ProcessDied watcheePid reason))
3
4  applyMonitorAction ns watcherPid watcheePid Succumb _ = do
5    tids <- readIORef (threadIds ns)
6    case Map.lookup watcherPid tids of
7      Just tid -> throwTo tid (MonitoredProcessDied watcheePid)
8      Nothing -> return ()

```

A monitor registered with `TrapExit` will receive the notification via a `ProcessDied` message in its mailbox. The monitor process can then observe the termination and decide what to do. Supervisor processes use this behaviour to figure out whether a process should be restarted or not. The `Succumb` monitor action will make the monitor receive an asynchronous exception, making it die together with the watched process. This is useful to tie together the lifetimes of processes that are meant to exist at the same time.

Node disconnection is treated as process failure. If an open socket is closed the surviving nodes will assume that any monitored process on that node has terminated. The surviving nodes will consider any monitored process as terminated with an `ExitOther` reason. This makes the running processes able to detect network failure as if it were ordinary process failure.

Explicit `exit` signals use the same path. If the target process is running on the current node, the node raises a `ProcessExit` exception in its thread, whereas if the process is remote, the node instead sends a `WireExit` command to the node that owns the process. In either case, the target process eventually runs the same finaliser.

4 Evaluation

We evaluate CloudMicroHaskell using two benchmarks and a larger case study. The two benchmarks aim to establish how much time it takes to spawn processes and send messages, as well as measure how much a workload can be sped up by using a cluster of interconnected machines on a network.

4.1 Benchmarks

Ring Benchmark. We take inspiration from Erlang with our first evaluation. A classic Erlang example is to spawn a *ring* of processes, where a message is sent from one process to another, through the entire ring, until the message arrives again at the original process. We use the ring of processes to measure how long it takes, on average, to spawn a single worker, and how long it takes to send a message from one process to another.

All processes in the ring are spawned on the same node. Node-local `spawn` and `send` usually don't go through the serialisation/deserialisation step, as there is no need for it if the function or data does not cross the network. We can instrument CloudMicroHaskell to force serialisation and deserialisation for these operations even on node-local operations. We measure the performance both with and without serialisation enabled, to gain some insight into how much time encoding and decoding the graph takes. The serialised process is roughly 32 kilobytes of data, whereas the serialised message sent through the ring is roughly 300 bytes.

We run this experiment on a laptop with an Intel i5-8365U CPU, and 16 GB of RAM. The results can be seen in table 1.

Table 1. Ring benchmark: per-process spawn and per-hop message time (ms, mean of 10 runs).

n	CloudMicroHaskell		CloudMicroHaskell serialised		Erlang/OTP	
	spawn	hop	spawn	hop	spawn	hop
100	2.077	2.123	2.869	2.304	0.0010	n/a
1,000	2.403	2.123	3.243	2.555	0.0011	0.0005
3,000	2.402	2.198	2.964	2.399	0.0014	0.0005
10,000	2.410	2.276	3.117	2.322	0.0013	0.0004

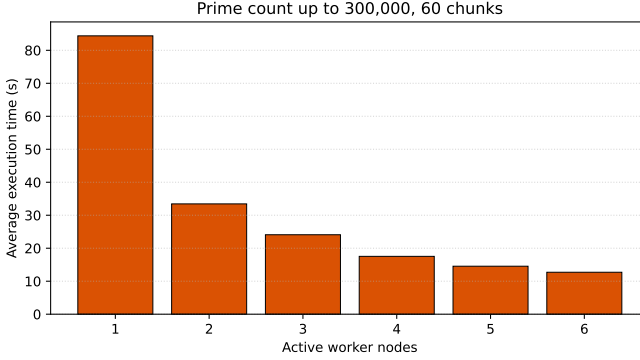


Fig. 2. Distributed work-pool benchmark for counting primes up to 300,000, split into 60 work units. Bars show mean execution time over 10 runs on one to six Google Cloud E2-standard-2 worker nodes; runtime drops from 82.5 s to 12.6 s, a 6.5 $\times$ speedup over the one-node baseline.

Distributed Work Pool Benchmark. This benchmark measures whether adding CloudMicroHaskell nodes yields proportional speedup on an embarrassingly parallel workload. We count the primes in a fixed integer interval, dividing it into equally sized segments that the work pool distributes as units of work across remote workers.

We distribute the workload over 6 remote Google Cloud E2-standard-2 machines, each equipped with 8 GB of RAM and 2 vCPUs. Each instance runs a CloudMicroHaskell node; these nodes are connected together. We run the workload using between one and six nodes, averaging over 10 results. The results can be seen in fig. 2.

4.2 Case Study

To illustrate the programming model beyond small code snippets and toy examples, we implement a file syncing application, MicroSync. The case study is modest in scope, but deliberately nontrivial. Several machines work together to keep the contents of a directory in sync, discover each other, broadcast local edits, exchange metadata, exchange partial chunks of files, and recover from failing helper processes. The case study shows that this application is expressed fairly naturally as a collection of communicating processes.

We dive straight in, and illustrate how an edit of a local file on machine A is advertised and applied to a remote copy of the same file on machine B; fig. 3 shows the corresponding message flow.

On machine A, a user modifies a local file. The *modify detector* process assigned to monitor that particular file detects this edit by observing the last modified timestamp, and reports this fact to its parent *file tracker* process. The file tracker process is MicroSync’s designated owner of one tracked file. It receives the message from the

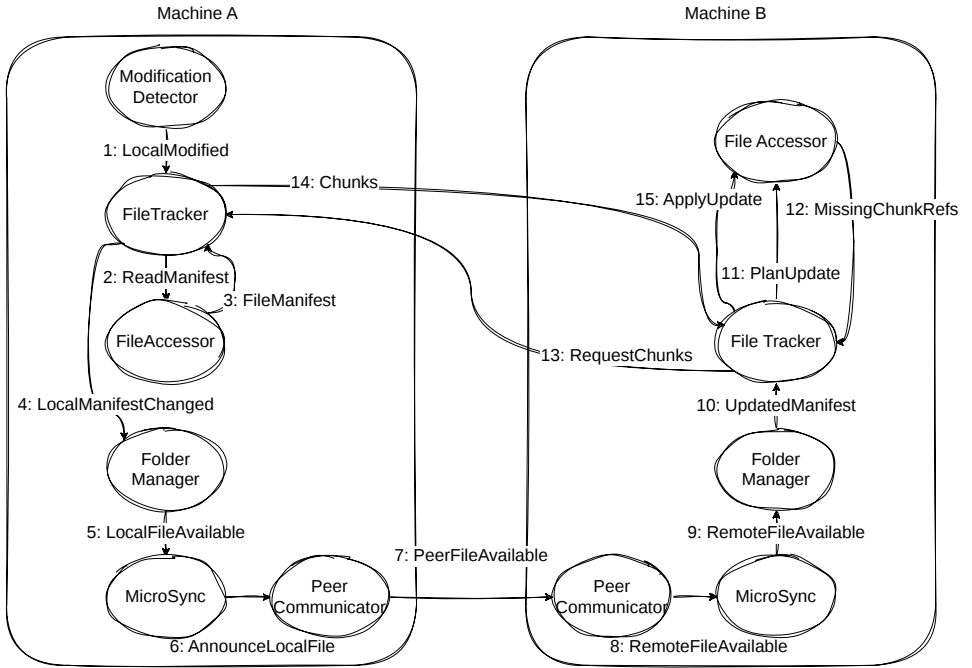


Fig. 3. Message flow for synchronising a local edit from machine A to machine B. The numbered arrows show the messages described in the walkthrough: the update is first advertised through the peer processes, after which the receiving file tracker requests missing chunks directly from the file tracker that advertised the version.

modification detector that the file has likely been modified, and asks the *file accessor* for the file manifest. A file accessor process is the only process that is allowed to modify a file on the disk, and is implemented as a server.

A file manifest is a compact description of a file version. It records, among other things, the path to the file, the file size, and most importantly, a list of chunk references. Each chunk reference contains an offset into the file, the size of the chunk, and a hash of the chunk contents. MicroSync uses *Content-Defined Chunking* to divide a file into chunks. The chunks are chosen based on the contents of the file, rather than using fixed byte offsets. The effect of this is that a small edit tends to change only chunks close to where the edit happened, allowing a remote MicroSync instance to request only the chunks that it does not already have.

Once the file accessor has supplied the file manifest, the file tracker sends it to the *folder manager* process. The folder manager process is the designated owner of the entire folder being synchronised. For this local-modification case, its main role is to turn the tracker’s manifest into a file-version announcement that is sent

upwards, to the top-level *MicroSync* process. The top-level *MicroSync* process routes the announcement to its *peer communicator* process. The peer communicator process informs the remote peer communicator on machine B what machine A's version of the file looks like. On machine B, the information flows in a similar way from the peer to the *MicroSync* top-level process, from there to the folder manager, which will in turn inform the responsible file tracker process that there exists a potentially new version of the file.

The file tracker on machine B will ask its file accessor for the local file manifest, figure out that the version on machine A is newer, and then compute which chunks of the file it should request from machine A. The broadcast message from machine A included the process identifier of the specific file tracker on machine A, so the equivalent file tracker on machine B will now request the missing chunks directly from the file tracker on machine A. This process identifier is only assumed valid for this one exchange, and is not retained on machine B for subsequent use. Once the chunks have been delivered, the file tracker on machine B instructs its file accessor to rebuild the file according to the new file manifest. It will supply the new chunks, and together with the chunks that already exist on machine B, the file is updated to have the same contents as the version on machine A.

fig. 3 shows two machines for clarity. However, in general, a *MicroSync* instance may be connected to several remote peers. The peer process broadcasts file-version announcements to all of them, and each receiving instance independently decides how to react to the announcement. The described application is built entirely from *CloudMicroHaskell* primitives such as named processes for discovery, message passing for routing, generic servers for stateful services, supervisors for recovery, etc. The following paragraph gives some details on the implementation, but naturally omits most of the application code.

The Process Tree. We have already touched upon several processes that are part of a *MicroSync* instance, but here we give a complete description of the different kinds of processes. A *MicroSync* instance is implemented as a process tree, with the top level process being the *MicroSync* process. It registers itself under the name "*microsync*" and starts a supervisor for the long-running services of the application. The supervisor uses a one-for-one strategy to permanently supervise four child processes. The child processes in question are the *Event Logger* process, the *Folder Manager* process, the *Peer Communication* process, and finally the *Peer Connector* process.

The event logger is a generic server, registered under the name "*eventlogger*", whose service is to accept events and log them to a persistent file on disk. It gives the rest of the application a single named process which events can be sent to for logging.

The peer connector process is responsible for discovering remote *MicroSync* instances. It periodically checks connected *CloudMicroHaskell* nodes to see whether a registered "*peer*" process exists.

```

1 discoverPeers :: ProcessM [Pid]
2 discoverPeers = do
3   localNode <- node
4   allNodes <- nodes
5   let remoteNodes = filter (/= localNode) allNodes
6
7   mPeers <- mapM (\n -> call n (whois "peer")) remoteNodes
8
9   return (nub (catMaybes mPeers))

```

This peer process is the control-plane process for remote MicroSync instances. It maintains a state identifying which connected peers exist, monitors them for disconnection, and forwards file-version announcements between remote machines.

The folder manager process owns the state of the synchronised folder. It manages the set of files being tracked at a point in time, and starts or stops file tracker processes as necessary. Additionally, it manages a folder scanner process, which scans the folder for new and deleted files, while each tracked file is represented by a dynamically added file tracker process.

A file tracker process is created dynamically for each tracked file. It owns the synchronisation protocol for that file and presents the rest of the application with a single process to address. The tracker itself is implemented as a generic server, registered under the file path, and it starts a private supervision tree for the helper processes associated with the file. Its modify detector reports suspected local changes, while its file accessor serialises all reads and writes to the underlying file.

fig. 4 illustrates how the processes are structured, and who supervises whom.

5 Discussion

Benchmarks. The results show that the cost of spawning processes remains close to constant, regardless of how many we spawn. Similarly, the time taken to send a single message from one process to another remains constant as well. If we make CloudMicroHaskell serialise spawned processes and sent messages even when they stay within the same node, we observe a slow-down in performance. What we observe is the added cost of serialisation and deserialisation. Spawning a process becomes 0.5 - 0.8 ms slower, while sending a message is 0.05 - 0.3 ms slower. The process closure, when serialised, took up 32 kB. The serialised graph is emitted in a textual ASCII format, which is not particularly compact.

We must also address the fact that MicroHaskell is orders of magnitude slower than Erlang when running the ring benchmark. Similarly, while not presented in the paper, we report that counting primes with GHC is two orders of magnitude faster than doing it with MicroHaskell. Both Erlang and GHC are compilers that have been tuned for performance over several decades by now, whereas MicroHaskell is a compiler in its infancy, not applying any optimisations to the code before emitting the combinator graph. The advantage of MicroHaskell over e.g. GHC is primarily its

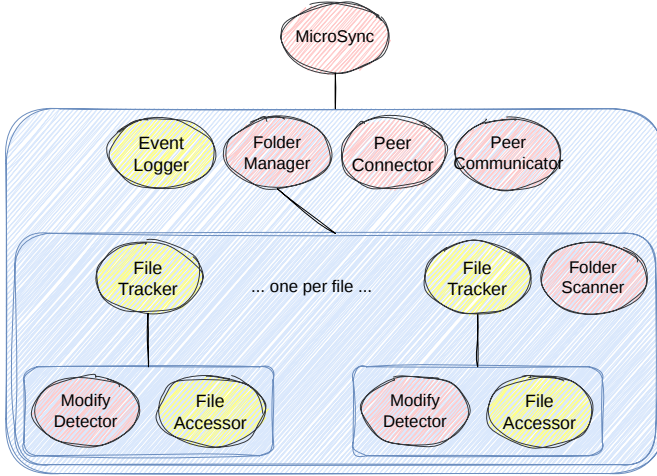


Fig. 4. Process tree of one MicroSync instance. Blue containers are supervisors, yellow processes are generic servers, and red processes are ordinary processes. File trackers are created dynamically, one per tracked file, and each tracker supervises its own modify detector and file accessor.

smaller and portable runtime, enabling deployment of Haskell programs on platforms such as microcontrollers and trusted execution environments, like Arm Cortex-M33 TrustZone-M. If a programmer wants high-performance Haskell, they should not use MicroHaskell, or perhaps offload the heavy computations to optimised C code using the foreign function interface.

The distributed prime counter shows significant speedups compared to sequential runtime. We point out that there seem to be a superlinear speedup when going from one to two remote machines, and we are not entirely sure why. The remote instances are separate instances, but it is impossible to say precisely where they are hosted and what else is hosted on the same hardware. While we cannot say for sure, we speculate that the phenomenon may be due to caching, and that using more than one MicroHaskell instance reduces the frequency with which the garbage collector has to run.

MicroSync. The purpose of MicroSync is not to evaluate file synchronisation performance, but to test whether the CloudMicroHaskell programming model is usable in a larger application. The case study exercises named processes, dynamic process creation, supervisors, monitors, generic servers, and cross-node message passing in one program. It therefore supports an expressiveness claim rather than a performance claim, namely that CloudMicroHaskell is sufficient to structure a nontrivial

distributed application while preserving the direct style illustrated in the smaller examples.

6 Related Work

Cloud Haskell. The work described in this paper is strongly related to Cloud Haskell. Cloud Haskell presented an elegant API, as evidenced by the work described in this paper reusing many parts of it. The primary difference is the treatment of `spawn`, where Cloud Haskell required the programmer to manually apply lambda-lifting to their process bodies. Furthermore, the programmer had to apply explicit closure-conversion when spawning such functions, adding cognitive overhead. Cloud-MicroHaskell removes this source-level closure conversion step. A process body can be written directly at the call to `spawn`, including references to variables in scope, and the runtime serialises the reachable graph when the process crosses a node boundary.

Cloud Haskell explicitly discusses the style of implementation used by CloudMicroHaskell, where the compiler or runtime provides primitives for serialising arbitrary values, and argues against it. The concerns raised there still apply. Some runtime-owned values, such as open file handles, should not be serialisable. A programmer may also accidentally serialise a much larger graph than intended, or serialise an unevaluated computation whose work was expected to happen before transmission.

Glasgow Distributed Haskell. Glasgow Distributed Haskell [7] (GDH) was an earlier effort to extend Haskell for distributed programming. GDH preceded Cloud Haskell, but the general idea was the same, that a distributed application should be described as a collection of communicating processes. GDH was elegant and well-designed, but practical concerns led to the project slowing down. It required substantial modification to GHC's runtime system of the time (the GUM [12] runtime), which limited its adoption and maintainability.

Elixir. The de facto implementation of actor model concurrency is Erlang, a dynamically typed language. Elixir [11] is a language that is executed with the same underlying runtime system, but whose syntax is more approachable. At the fundamental level Elixir is not much different from Erlang, but it offers a more modern tooling and ecosystem.

Tierless and Choreographic Haskell. Recent work has explored several ways to describe distributed Haskell programs from a single source program. In tierless programming, the behaviours of multiple tiers are written and type-checked together, then compiled into separate executables. The framework of Ekblad and Claessen [3] follows this style for client-server web applications. The client and server are described in one program, and the compiler separates the code so that client code does not appear in the server, and vice versa.

The idea was later adopted by HasTEE [9], a framework for programming *Trusted Execution Environments* in Haskell. They observed that a trusted computing unit running on Intel SGX hardware assumes the role of a server, and that the programming

model of Ekblad and Claessen [3] was a good fit because of its separation of client code and server code.

Choreographic programming [6] takes a related whole-program view, but describes communication protocols globally rather than from one endpoint’s perspective. A communication action is written once, naming both sender and receiver. When the choreography is interpreted or projected for a particular endpoint, that action becomes a send, a receive, or no local action. HasChor [10] embeds this style of choreographic programming in Haskell. Its presentation is elegant and simple, but the original implementation produced executables that contained the code for every endpoint in the network. Krook and Hammersberg [5] address this limitation by specialising HasChor programs with respect to a known endpoint, producing executables that contain only the code needed by that endpoint.

7 Conclusions & Future Work

We set out to explore what the Cloud Haskell programming model looks like when serialisation is provided by the runtime system rather than exposed as `Closure` and `Serializable` constraints in the source-level API. CloudMicroHaskell shows that this design point is practical: remote processes can be spawned in direct style, messages can contain ordinary values and functions, and familiar Erlang-style abstractions such as generic servers and supervisors can be implemented as ordinary libraries.

The benefit is ergonomic. Programs can often be written in the style one would write for a local collection of communicating processes, while the runtime serialises the reachable MicroHaskell graph when a computation or message crosses a node boundary. The cost is that the boundary has not disappeared, but moved. Deserialisation relies on the receiver’s expected type matching the graph that was sent, and some mistakes that Cloud Haskell exposes in the source-level API become runtime failures in CloudMicroHaskell. Programmers must also be aware of runtime-owned resources, such as `MVars` and open file handles, and of laziness, meaning that sending an unevaluated graph may move work to an unintended node.

There are several natural ways to improve this design. First, CloudMicroHaskell currently serialises the transitive closure of the graph being sent. This is simple, but may duplicate code that both nodes already have. A future system could establish a shared vocabulary of known code and serialise references to those definitions, while still falling back to graph serialisation for genuinely dynamic code and environments.

Second, the node runtime is currently itself a Haskell program. Moving parts of the node implementation into the MicroHaskell runtime, or exposing lower-level primitives for spawning and message delivery, could reduce overhead and shrink the graphs that must be transmitted. This would also give a cleaner place to enforce dynamic checks around graph types, serialisable runtime resources, and version compatibility between nodes.

Acknowledgments

The authors would like to thank Koen Claessen, Mary Sheeran, John Hughes, and Simon Peyton-Jones for their helpful discussions and feedback.

References

- [1] Lennart Augustsson. 2011. Keynote: Pragmatic Haskell. In *Proceedings of the 8th ACM SIGPLAN Workshop on Commercial Users of Functional Programming (CUPP)*. ACM, Tokyo, Japan. Discussions on the design and deployment of the Mu language at Standard Chartered Bank.
- [2] Lennart Augustsson. 2024. MicroHs: A small compiler for Haskell. In *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium*. 120–124.
- [3] Anton Eklblad and Koen Claessen. 2014. A seamless, client-centric programming model for type safe web applications. In *Proceedings of the 2014 ACM SIGPLAN symposium on Haskell, Gothenburg, Sweden, September 4-5, 2014*, Wouter Swierstra (Ed.). ACM, 79–89. <https://doi.org/10.1145/2633357.2633367>
- [4] Jeff Epstein, Andrew P. Black, and Simon L. Peyton Jones. 2011. Towards Haskell in the cloud. In *Proceedings of the 4th ACM SIGPLAN Symposium on Haskell, Haskell 2011, Tokyo, Japan, 22 September 2011*, Koen Claessen (Ed.). ACM, 118–129. <https://doi.org/10.1145/2034675.2034690>
- [5] Robert Krook and Samuel Hammersberg. 2024. Welcome to the Parti(tioning) (Functional Pearl): Using Rewrite Rules and Specialisation to Partition Haskell Programs. In *Proceedings of the 17th ACM SIGPLAN International Haskell Symposium, Haskell 2024, Milan, Italy, September 6-7, 2024*, Niki Vazou and J. Garrett Morris (Eds.). ACM, 27–40. <https://doi.org/10.1145/3677999.3678276>
- [6] Fabrizio Montesi. 2014. Choreographic programming. (2014).
- [7] Robert F. Pointon, Philip W. Trinder, and Hans-Wolfgang Loidl. 2000. The Design and Implementation of Glasgow Distributed Haskell. In *Implementation of Functional Languages, 12th International Workshop, IFL 2000, Aachen, Germany, September 4-7, 2000, Selected Papers (Lecture Notes in Computer Science)*, Markus Mohnen and Pieter W. M. Koopman (Eds.). Springer, 53–70. https://doi.org/10.1007/3-540-45361-X_4
- [8] Eric Rescorla. 2001. *SSL and TLS: Designing and Building Secure Systems*. Addison-Wesley Professional.
- [9] Abhiroop Sarkar, Robert Krook, Alejandro Russo, and Koen Claessen. 2023. HasTEE: Programming Trusted Execution Environments with Haskell. In *Proceedings of the 16th ACM SIGPLAN International Haskell Symposium, Haskell 2023, Seattle, WA, USA, September 8-9, 2023*, Trevor L. McDonell and Niki Vazou (Eds.). ACM, 72–88. <https://doi.org/10.1145/3609026.3609731>
- [10] Gan Shen, Shun Kashiwa, and Lindsey Kuper. 2023. HasChor: Functional Choreographic Programming for All (Functional Pearl). *Proc. ACM Program. Lang.* 7, ICFP (2023), 541–565. <https://doi.org/10.1145/3607849>
- [11] The Elixir Team. 2026. The Elixir Programming Language. <https://elixir-lang.org/> Accessed: 2026-05-21.
- [12] Philip W Trinder, Kevin Hammond, James S Mattson Jr, Andrew S Partridge, and Simon L Peyton Jones. 1996. GUM: a portable parallel implementation of Haskell. *ACM SIGPLAN Notices* 31, 5 (1996), 79–88.
- [13] D. A. Turner. 1979. A new implementation technique for applicative languages. *Software: Practice and Experience* 9, 1 (1979), 31–49. <https://doi.org/10.1002/spe.4380090105>

A API Reference

This appendix lists the core CloudMicroHaskell API used throughout the paper.

```

1  data Node a
2  data NodeId
3  data NodeConfig -- = (ip, port)
4  data ProcessM a; instance MonadIO ProcessM
5  data Pid
6  runNode :: NodeConfig -> Node () -> IO ()

```

A node says *where* something is: a location in a network. When a node connects to an existing network of nodes, they form a mesh, whereby every node is connected to every other node.

```

1  connect :: NodeConfig -> ProcessM ()

```

Each node can host zero or more processes, which are independent computation units. Processes (almost) don't share any memory with other processes, with the exception(s) described in section 3. There are some basic operations for processes to identify themselves, their node, the nodes that are connected to the network, and to terminate.

```

1  self      :: ProcessM Pid
2  node      :: ProcessM NodeId
3  nodes     :: ProcessM [NodeId]
4  terminate :: ProcessM a

```

A process that wishes to retrieve a list of every node in the network, aside from its own host node, may execute

```

1  do mynode <- node
2     allnodes <- nodes
3     return $ filter (/= mynode) allnodes

```

Processes are started either by running an initial process on a node, or by spawning a new process from an existing one. A derived `call` operation spawns a process and waits for its result.

```

1  runProcessM :: ProcessM () -> Node ()
2  spawn :: NodeId -> ProcessM () -> ProcessM Pid
3  call :: NodeId -> ProcessM a -> ProcessM a

```

As the processes are independent and don't share any memory, they cannot perform any meaningful task without interacting with each other. Interactions are done via message passing, where processes can `send` and `expect` messages to and from one another. `send` is an asynchronous call that returns immediately, whereas `expect` is a synchronous call that doesn't return until it has received a message of the correct type. It is appropriate to think of it as if every process had a mailbox where messages

of any type can go in, and where `expect` goes through the letters in the order they arrived, looking for one of the right type.

```
1  send  :: Identifiable process => process -> a -> ProcessM ()
2  expect :: ProcessM a
```

For selective receive, CloudMicroHaskell follows the matcher interface introduced by Cloud Haskell [4]. A matcher specifies one acceptable message type, optionally guarded by a predicate, and `receiveWait` blocks until any matcher succeeds. There is additionally a `matchUnknown` handler that will match a message of any type, removing it from the mailbox.

```
1  data MatchM q a
2  match      :: (a -> ProcessM q) -> MatchM q ()
3  matchIf    :: (a -> Bool) -> (a -> ProcessM a) -> MatchM q ()
4  matchUnknown :: ProcessM q -> MatchM q ()
5  receiveWait :: [MatchM q ()] -> ProcessM q
6  receiveTimeout :: Int -> [MatchM q ()] -> ProcessM (Maybe q)
```

A matcher is one branch of a selective receive. It recognises a message and runs a handler producing a value of type `q`. `receiveWait` blocks until one of the supplied matchers succeeds, while `receiveTimeout` bounds the wait and returns `Nothing` if no message matches in time.

For example, a process can wait for either a request to divide two numbers, or for a command to shut down

```
1  data Div      = Div Pid Int Int
2  data Shutdown = Shutdown
3
4  server :: ProcessM ()
5  server = receiveWait
6    [ matchIf (\(Div _ _ y) -> y /= 0) $ \(Div sender x y) ->
7      send sender (x `div` y)
8    , match $ \Shutdown -> terminate
9    ]
```

The two matchers receive messages of different types, but both branches are part of the same blocking receive. The first branch accepts only non-zero division requests and replies to the sender. The second branch accepts a shutdown command and terminates the process. Messages that do not match either branch remain in the mailbox.

So far, the API describes how live processes communicate. For fault-tolerant programs, processes must also be able to observe one another's lifetime, where a missing reply may mean that another process has terminated, crashed, or disappeared with its node. CloudMicroHaskell exposes this through monitors.

```
1  data ProcessDied = ProcessDied Pid ExitReason
```

```

2  data MonitorAction = TrapExit | Succumb
3
4  monitor :: MonitorAction -> Pid -> ProcessM ()

```

When process *p* monitors process *q*, *p* is notified if *q* terminates. With `TrapExit`, the notification is delivered as a `ProcessDied` message in *p*'s mailbox. With `Succumb`, *p* terminates as well.

Aside from observing process termination, processes can also cause process termination, using the `exit` function. This makes it possible to designate application-specific roles to processes, such as workers and supervisors. Supervisor processes might send an exit signal to a process that has entered an unhealthy state.

```

1  data ExitReason
2    = ExitNormal      -- ^ Process exited normally
3    | ExitShutdown    -- ^ Process exited upon request of another process, e.g. a supervisor
4    | ExitKill        -- ^ Process was forcibly killed, equivalent to kill -9.
5    | ExitOther String -- ^ Other custom reasons why a process was terminated
6
7  exit :: Pid -> ExitReason -> ProcessM ()

```

An example that illustrates how `monitor` and `exit` can be used to control behaviour is shown below

```

1  do child <- spawn n childBody
2
3      _ <- spawn n $ do
4          monitor TrapExit child
5          ProcessDied _ _ <- expect
6          liftIO (putStrLn "the child died!")
7
8      linked <- spawn n $ do
9          monitor Succumb child
10         expect
11
12     monitor TrapExit linked
13     exit child ExitKill
14     ProcessDied _ _ <- expect

```

The first monitor survives the child's death and receives a `ProcessDied` message. The second monitor uses `Succumb`, so it dies together with the child. By monitoring the second process with `TrapExit`, the parent observes this cascading termination as an ordinary mailbox message.

The process registry maps node-local names to process identifiers. In the example above, the two monitoring processes had to have a way of knowing the process identifier of the child process. Sometimes it is not ergonomic to distribute process identifiers to everyone who has to know them. Furthermore, a process may be

restarted by a supervisor process, in which case the pre-distributed process identifier will no longer be valid. The process registry lets a process register itself under a name on the node it is running on.

```
1 register :: Pid -> String -> ProcessM ()
2 unregister :: String -> ProcessM ()
3 whois     :: String -> ProcessM (Maybe Pid)
```

Since `String` is an instance of `Identifiable`, `send` may target a registered name.

```
1 send :: Identifiable i => i -> a -> ProcessM ()
```

Types that are `Identifiable` are process identifiers and strings. If a message is sent to a named process, the process identifier is fetched from the process registry before the message is sent. If the name is not registered, the `send` fails with an exception, much like it would in Erlang. We illustrate the registry with a simple example

```
1 do n <- node
2
3   _ <- spawn n $ do
4     s <- self
5     register s "child"
6     str <- expect
7     liftIO $ putStrLn str
8
9     send "child" "hello"
```

The parent process spawns a child process on the local node, which registers itself under the name `"child"`. The parent process then sends a message to the spawned process simply by referring to it by its name. We emphasise that the name registry is node-local, meaning that messages can only be sent to named processes on the same node. The process identifier of a remote named process can be retrieved without much fuss, however. We can simply issue a synchronous call, reading the remote registry

```
1 remoteProcess <- call n' (whois "child") -- n' is the NodeId of some remote node
2 case remoteProcess of
3   Just pid -> send pid "hello"
4   Nothing  -> return ()
```